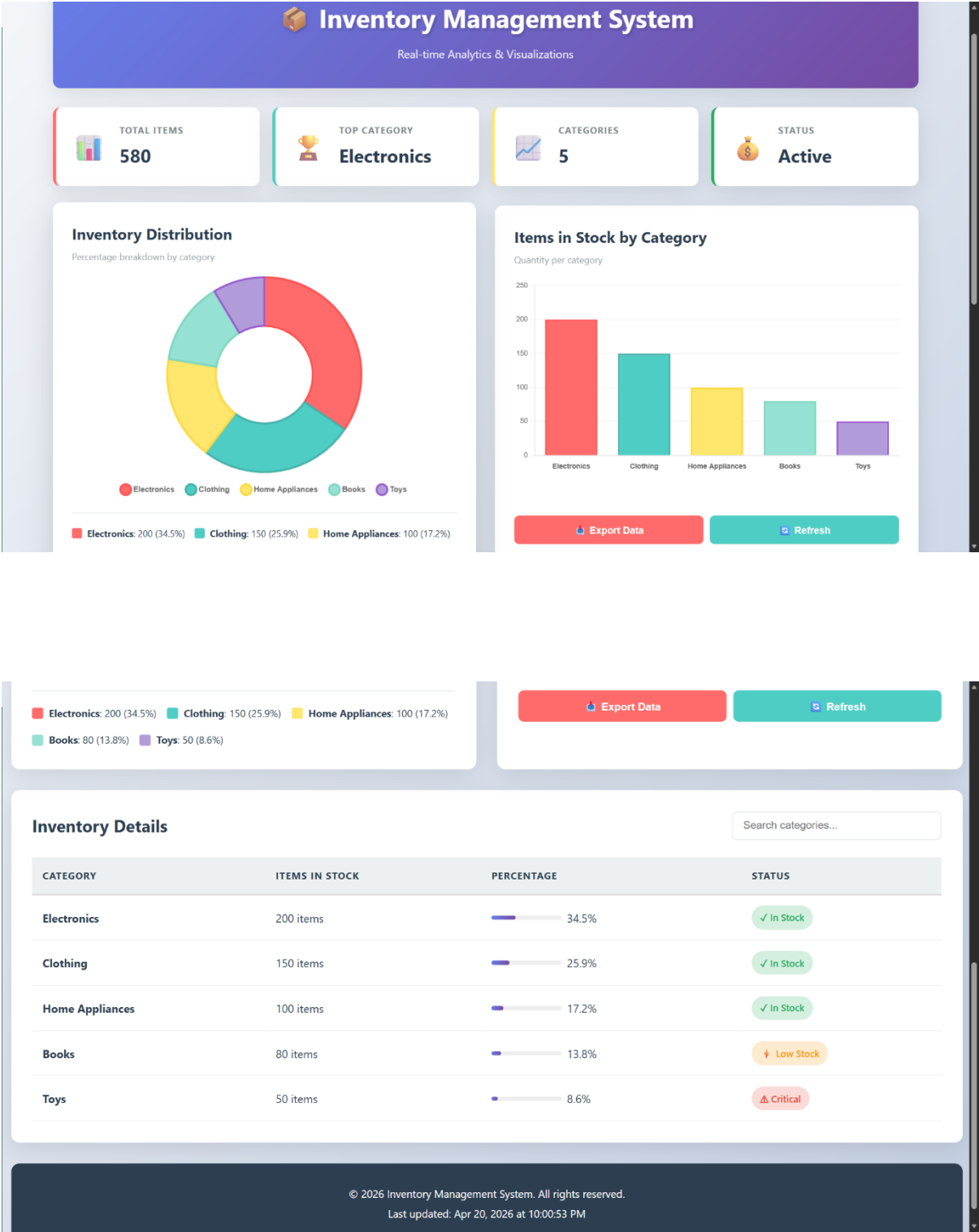




Step 1: Build the Dashboard Structure (index.html)

This file sets up the grid layout, imports Chart.js, and creates the containers for the KPI cards, charts, and data table.

```
index.html X
index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Inventory Management System</title>
7      <link rel="stylesheet" href="styles.css">
8      <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
9  </head>
10 <body>
11     <div class="container">
12         <!-- Header -->
13         <header class="header">
14             <div class="header-content">
15                 <h1>📦 Inventory Management System</h1>
16                 <p>Real-time Analytics & Visualizations</p>
17             </div>
18         </header>
19
20         <!-- Stats Cards -->
21         <section class="stats-section">
22             <div class="stat-card">
23                 <div class="stat-icon">📊</div>
24                 <div class="stat-info">
25                     <h3>Total Items</h3>
26                     <p id="totalItems" class="stat-value">580</p>
27                 </div>
28             </div>
29             <div class="stat-card">
30                 <div class="stat-icon">📦</div>
31                 <div class="stat-info">
32                     <h3>Top Category</h3>
33                     <p id="topCategory" class="stat-value">Electronics</p>
34                 </div>
35             </div>
36             <div class="stat-card">
37                 <div class="stat-icon">📄</div>
```

Step 2: Apply Modern UI Styling (styles.css)

This CSS provides the refined, cloud-dashboard aesthetic requested, utilizing flexbox/grid for responsive alignment and specific color-coding for status indicators.

```
# styles.css X
# styles.css > 
1 * {
2   margin: 0;
3   padding: 0;
4   box-sizing: border-box;
5 }
6
7 :root {
8   --primary-color: #FF6868;
9   --secondary-color: #4ECDC4;
10  --accent-color: #FFE66D;
11  --dark-color: #2C3E50;
12  --light-color: #ECF0F1;
13  --success-color: #27AE60;
14  --danger-color: #E74C3C;
15  --warning-color: #F39C12;
16  --bg-gradient: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
17  --border-radius: 12px;
18  --box-shadow: 0 8px 32px rgba(0, 0, 0, 0.1);
19  --transition: all 0.3s ease;
20 }
21
22 body {
23   font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
24   background: linear-gradient(135deg, #f5f7fa 0%, #a3c3fe 100%);
25   color: var(--dark-color);
26   line-height: 1.6;
27   min-height: 100vh;
28 }
29
30 .container {
31   max-width: 1400px;
32   margin: 0 auto;
33   padding: 20px;
34 }
35
36 /* Header */
37 .header {
```

Step 3: Implement Visualization & Logic (script.js)

This script initializes the charts and dynamically generates the HTML for the data table based on the inventory levels, calculating percentages and assigning the correct status badges automatically.

```
js script.js X
js script.js > ...
1 // Inventory Data
2 const inventoryData = {
3   labels: ['Electronics', 'Clothing', 'Home Appliances', 'Books', 'Toys'],
4   datasets: [{
5     label: 'Items in Stock',
6     data: [200, 150, 100, 80, 50],
7     backgroundColor: [
8       '#FF686B',
9       '#4ECDC4',
10      '#FFE66D',
11      '#95E1D3',
12      '#B19CD9'
13    ],
14    borderColor: [
15      '#FF5252',
16      '#4583AA',
17      '#FFD93D',
18      '#7FD8BE',
19      '#9D5CD0'
20    ],
21    borderWidth: 3
22  }]
23 };
24
25 let pieChartInstance = null;
26 let barChartInstance = null;
27
28 // Initialize on page load
29 document.addEventListener('DOMContentLoaded', () => {
30   initializePage();
31 });
32
33 function initializePage() {
34   // Create Charts
35   createPieChart();
36   createBarChart();
37 }
```